

Agentic RAG

בניית והערכת סוכן RAG עם קריאה לכלים

פרויקט סופי

תאריך פרסום	4 בספטמבר 2026	מועד הגשה	ייתבצע במודול הקורס
אופן העבודה	אישי	משקל בציון הקורס	30% - פרויקט סופי + הגנה בעל פה, כמוגדר בדף המידע של הקורס
פורמט הגשה	Notebook/קוד הרץ מתחילתו ועד סופו + דו"ח PDF (8-10 עמודים, לא כולל נספחים) + קובץ requirements/תא גרסאות + README קצר + הגנה בעל פה על הפרויקט		
סביבת חישוב	לא נדרש אימון מודל וללא צורך ב-GPU ייעודי. כל החלקים ניתנים למימוש מול API של מודלי שפה (פתוחים ו/או מסחריים) או מודלים פתוחים מקומיים קטנים. ר' תיבת "אילוצי משאבים" בהמשך.		

אילוצי משאבים - תקציב API וזמן ריצה (קריאה חובה לפני תחילת העבודה)

הפרויקט אינו כולל אימון או fine-tuning של מודל - המיקוד הוא בבניית pipeline של RAG ובשכבת סוכן מעליו, לא באימון. כל המודלים המעורבים (agent, judge) נצרכים דרך inference בלבד. קורפוס: 50-300 מסמכים/יחידות טקסט (מאמרים, תיעוד טכני, פרקי ספר, מדיניות פנים-ארגונית מדומה וכד') בתחום מוגדר וממוקד - מספיק כדי להדגים אתגרי retrieval אמיתיים, לא Wikipedia שלמה ולא מאגר עצום. תכנון מראש כמות שאלות/משימות בדיקה (סעיפים א'-ו' להלן) כך שכלל הריצות (retrieval, generation, agent, judge) ייכנסו במכסת ה-API/compute שברשותכם. מותר להשתמש במודל מסחרי (API) הן לסוכן והן ל-judge, ובלבד שה-judge ממשפחת מודלים שונה מהמודל הנבדק - בדיוק כפי שנדרש במטלות הקודמות. תעדו כל קריאת API (פרומפט, תשובה, טוקנים משוערים) כדי לאפשר שחזור וניתוח כשלים בדיעבד.

רקע ומטרה

בהרצאות 7-8 למדתם כיצד RAG מרחיב מודל שפה בידע חיצוני ועדכני שלא היה חלק מהאימון, כיצד סוכנים משתמשים בכלים (function calling / ReAct) כדי לפעול מעבר לטקסט בלבד, ואילו כשלים ספציפיים מאפיינים מערכות מסוג זה - משגיאות retrieval, דרך כשלי חיזוי והפעלה של כלים, ועד כשלי סינתזת תשובה סופית.

במטלות 1-2 התמקדתם במודל שפה כשלעצמו - תחילה כ"קופסה שחורה", ולאחר מכן באימונו ויישורו. בפרויקט הסופי תבנו ותעריכו מערכת שלמה יותר: pipeline של RAG מעל קורפוס שתבחרו, ומעליו שכבת סוכן המפעילה כלים כדי לבצע משימות רב-שלביות. הדגש, כמו לאורך הקורס, הוא על עיצוב ניסוי מבוקר, מדידה כמותית של כשלים לפי הטקסונומיה המדויקת שנלמדה בהרצאה 8, והסקת מסקנות מכוילות - לא על "איזו מערכת הכי טובה".

חלק א' - קורפוס ו-Retrieval Pipeline

- בחרו דומיין ממוקד ובנו קורפוס בגודל 50-300 מסמכים/יחידות (למשל: תיעוד טכני של ספרייה, אוסף מאמרים בנושא אחד, מדריך פנים-ארגוני מדומה).
- ממשו pipeline retrieval דו-שלבי: (1) candidate retrieval- embeddings ממנו מייצגים, ואופציונלית גם שיטה מבוססת מילות מפתח reranking (2); (BM25); של המועמדים (LLM-based או cross-encoder reranker).
- תעדו במדויק את כל ההיפרפרמטרים: embedding model, chunking, overlap, chunk, bins k-ו לפני/אחרי reranking.

- הגדירו סט קבוע של 15–20 שאליות בדיקה (יישמר קבוע לאורך שאר הפרויקט) ובדקו ידנית את relevance@k של המסמכים שהוחזרו עבור כל שאלית.

שאלת ניתוח: מה השפיע יותר על איכות ה-retrieval- אסטרטגיית ה-chunking או שלב ה-reranking? האם יש שאליות שבהן ה-retrieval נכשל לגמרי ($recall=0$), ומה אפיון אותן?

חלק ב'- Generation מבוסס-הקשר (Grounded Generation)

- בנו baseline של RAG: הזינו למודל את המסמכים שהוחזרו יחד עם השאלית, וקבלו תשובה סופית.
- הגדירו קריטריון groundedness ברור: כל טענה עובדתית בתשובה חייבת להיות נתמכת (ניתנת לאיתור) באחד המסמכים שהוחזרו.
- הריצו את אותו סט שאליות גם ללא retrieval כלל (LLM בלבד, "closed-book") - ריצה זו תשמש בייסליין להשוואה בחלק ו'.

שאלת ניתוח: באילו מקרים ה-RAG baseline "המציא" מידע (הזיה) על אף שמסמכים רלוונטיים היו זמינים בהקשר? האם ניתן לייחס זאת לאיכות ה-retrieval, או לחולשת המודל בסינתוז מידע נתון?

חלק ג'- שכבת סוכן: Function Calling / ReAct

- הגדירו לפחות 2–3 כלים (למשל: כלי חיפוש בקורפוס מעל ה-pipeline מחלק א', מחשבון, כלי לשליפת מידע מובנה כמו טבלה/JSON, כלי API חיצוני כגון תאריך נוכחי או מזג אוויר).
- ממשו את הסוכן בשתי גישות מקבילות: (1) function calling חד-שלבי דרך ה-API הרשמי של הספק; (2) לולאת (ReAct (thought → action → observation עם prompting ידני.
- הגדירו סט קבוע של 10 משימות בדיקה (יישמר קבוע לאורך שאר הפרויקט), מתוכן לפחות 5 הדורשות שרשרת רב-שלבית אמיתית של קריאות לכלים - לא קריאה בודדת.

שאלת ניתוח: אילו סוגי משימות דרשו שרשור נכון של כמה כלים ברצף? היכן הסוכן "התבלבל" בסדר הפעולות או קרא לכלי לפני שהיה לו מידע מספיק?

חלק ד'- הערכת כשלי סוכן לפי הטקסונומיה מהרצאה 8

- על סט המשימות הקבוע מחלק ג', תייגו ידנית כל ריצה כושלת (על פני שתי גישות הסוכן) לפי הטקסונומיה המדויקת מהרצאה 8:
 1. שגיאת חיזוי כלי (Tool prediction error): אינו משתמש בכלי (no-tool) / מזה כלי שלא קיים (hallucinated-tool) / משתמש בכלי הלא נכון (wrong-tool) / מזין ארגומנט שגוי (wrong-argument).
 2. שגיאת קריאה לכלי (Tool call error): הכלי מחזיר תשובה שגויה/שגיאה (wrong-response) / הכלי אינו מחזיר דבר (no-response).
 3. שגיאת יצירת תשובה (Response generation error): התשובה הסופית אינה משקפת נכונה את תשובת הכלי (wrong-response), גם כשהכלי עצמו הצליח.
- בנו טבלת תדירויות המשווה בין סוגי הכשלים שנצפו בכל אחת משתי גישות הסוכן (function calling מול ReAct).

שאלת ניתוח: איזה סוג כשל היה הנפוץ ביותר בכל גישה? האם ReAct סבל יותר משגיאות חיזוי כלי (בשל חופש רב יותר בפורמט הפלט) לעומת function calling הרשמי, או שהתמונה הפוכה?

חלק ה'- LLM-as-a-Judge להערכת Groundedness

- ממשו judge (ממשפחת מודלים שונה מהמודל הנבדק) הבודק groundedness/factuality: לכל תשובה, בדקו האם כל טענה בה נתמכת במסמכים שהוחזרו ו/או בתוצאות הכלים שהתקבלו, בפלט structured הכולל נימוק (rationale) ולאחריו ציון.
- הריצו את ה-judge על אותו מדגם פעמיים (למשל בשינוי סדר הראיות המוצגות לו) ומדדו את העקביות הפנימית שלו.
- על מדגם של 20–30 דוגמאות, חשבו הסכמה בין תיוג ה-judge לבין תיוג ידני שלכם (למשל Cohen's kappa או אחוז הסכמה פשוט).

שאלת ניתוח: היכן ה-judge טעה ביחס לתיוג הידני? האם ניכרת הטיה לכיוון תשובות ארוכות ומשכנעות אך לא מבוססות (verboesity bias), בדומה למה שנלמד בהרצאה 8?

חלק ו'- אבלציה השוואתית

- השוו שלוש תנאים על אותו סט משימות קבוע (משילוב חלקים ב' ו-ג'): (1) LLM בלבד, ללא retrieval ו-לא כלים; (2) RAG בלבד (retrieval + generation, ללא שכבת סוכן/כלים); (3) הסוכן המלא (RAG + כלים).
- מדדו לכל תנאי: אחוז הצלחה (לפי תיוג ה-judge ו/או תיוג ידני), שיעור הזיה/groundedness, ויעילות (מספר קריאות למודל/לכלים, טוקנים, זמן תגובה).
- **שאלת ניתוח:** היכן הסוכן המלא השתפר הכי הרבה ביחס לבייסליינים? האם יש משימות שבהן RAG-בלבד הספיק, וה-agent רק הוסיף מורכבות וסיכון לכשלים נוספים בלי תועלת אמיתית?

סינתזה מסכמת- "פרופיל סוכן" (עמוד אחד)

כתבו עמוד סיכום הכולל:

- טבלה מסכמת: תנאי (LLM-בלבד / RAG-בלבד / סוכן מלא) מול מדדים (אחוז הצלחה, שיעור הזיה, יעילות).
- פילוח כשלי הסוכן לפי הטקסונומיה מחלק ד'- אילו סוגי כשל היו הדומיננטיים, ולמה.
- מה השתפר בוודאות סבירה, ומה תלוי-judge ולכן פחות ודאי.
- המלצה מנומקת: מתי הייתם ממליצים על RAG-בלבד לעומת סוכן מלא עם כלים בסביבת production, ולמה.

בנוס (עד 5 נקודות, בחרו אפשרות אחת בלבד)

- לולאת self-correction: הסוכן מזהה כשל (מה-judge או מכלל היוריסטי) ומנסה שוב עם תיקון, והשוואת שיעור ההצלחה לפני/אחרי הוספת הלולאה.
- השוואת retrieval צפוף (dense) מול BM25 מול hybrid, ומדידת ההשפעה שלה במורד הזרם על שיעור ה-groundedness של התשובה הסופית.
- פיצול planner/executor: מודל אחד מתכנן את רצף הכלים הנדרש, ומודל (או קריאה) נפרד מבצע כל צעד-השוואה מול סוכן end-to-end על אותו סט משימות.
- מבחן עמידות: הזרקת מסמכי-הסחה (distractors) לא רלוונטיים או סותרים לקורפוס, ומדידת ההשפעה על groundedness ושיעור ההזיה.
- בדיקת עמידות ה-judge עצמו: הרצת אותו judge פעמיים על אותן תשובות (או בסדר ראיות הפוך) ומדידת ה-agreement בין שתי הריצות.

נספח א'- דרישות שחזוריות (Reproducibility)

- תעדו לכל ריצה: מודל מדויק (name/revision) של כל LLM המעורב- agent, judge, embedding/reranker.
- קבעו seed בכל מקום שיש דגימה אקראית (בחירת קורפוס, sampling, decoding), ותעדו אותו.
- אין להסתמך על ריצה בודדת כאשר קיים רעש משמעותי (למשל אחוז הצלחה על מדגם קטן)- חזרו על מדידות רועשות.
- לכל גרף/טבלה- כותרת ברורה, תיוג צירים, ומקרא.
- הפרידו באופן ברור בדו"ח בין תוצאה אמפירית, פרשנות, וספקולציה.

הנחיות מחייבות ל-judge (חלק ה')

ה-judge חייב להיות ממשפחת מודלים שונה מהמודל הנבדק/הסוכן (למשל אם הסוכן מבוסס Qwen-ה-judge יכול להיות Llama, Gemma, או מודל API).
 פלט ה-judge חייב לכלול נימוק (rationale) לפני הציון- לא ציון בלבד.
 אם משנים את סדר הצגת הראיות/ההקשר ל-judge, דווחו האם השינוי משפיע על הציון- אל תבחרו את הכיוון הנוח.
 דווחו לא רק אחוז הסכמה עם התיוג הידני אלא גם את גודל המדגם, ואל תנפחו מסקנות מהפרשים קטנים.
 תעדו את הפרומפט המדויק ששימש כ-judge, כולל תבנית הפלט/schema אם השתמשתם ב-structured output.
 אם אותו judge משמש גם לתיוג הכשלים בחלק ד' וגם להערכת groundedness בחלק ה', חובה לציין זאת בדו"ח כמגבלה מתודולוגית מפורשת.

מדיניות כלי AI

מותר להשתמש בכלי AI לצורכי תכנות, debugging ושיפור ניסוח, אך האחריות להבנת הקוד, הניסויים והמסקנות היא שלכם- בדיוק כמו במטלות הקודמות.

רשימת בדיקה להגשה

- ☐ Notebook/קוד שרץ מתחילתו ועד סופו ללא שגיאות
- ☐ דו"ח PDF בן 8–10 עמודים (לא כולל נספחים)
- ☐ תיעוד מדויק של כל מודל/גרסה המעורב (agent, judge, embedding/reranker)
- ☐ seed מתועד בכל מקום עם דגימה אקראית
- ☐ קובץ requirements / תא גרסאות ספריות
- ☐ README קצר עם הוראות הרצה, שמות מודלים ודרישות סביבה
- ☐ נספח עם פלטים גולמיים מלאים (אופציונלי אך מומלץ)
- ☐ הכנה להגנה בעל פה על הפרויקט (ראו דף המידע של הקורס)

מחזון ציון (רכיב הפרויקט הכתוב/קוד)

קטגוריה	משקל	מה נבדק
עיצוב ניסוי ומתודולוגיה	20%	בקרת משתנים באבלציה בחלק ו', סט משימות/שאליות קבוע, שימוש נכון ב-LLM-as-a-Judge
ביצוע טכני	20%	קוד עובד, retrieval pipeline תקין, מימוש תקין של שתי גישות הסוכן, תיעוד היפרפרמטרים
כיסוי נושאי הקורס	20%	RAG (retrieval/chunking/reranking), function calling/ReAct, טקסונומית כשלי סוכן, LLM-as-a-Judge
ניתוח והסקת מסקנות	25%	מסקנות מכוילות, זיהוי מגבלות (עקביות ה-judge, גודל מדגם), פרשנות מול ספקולציה

איכות דו"ח והצגה	15%	בהירות, גרפים/טבלאות מתויגות, מבנה
------------------	-----	------------------------------------

* ההגנה בעל פה על הפרויקט נבחנת ומצוינת בנפרד, כמוגדר בדף המידע של הקורס.

בהצלחה! הפעם לא רק בודקים או מיישרים מודל בודד- בונים סביבו מערכת שלמה, ובודקים אותה כמו שצריך.